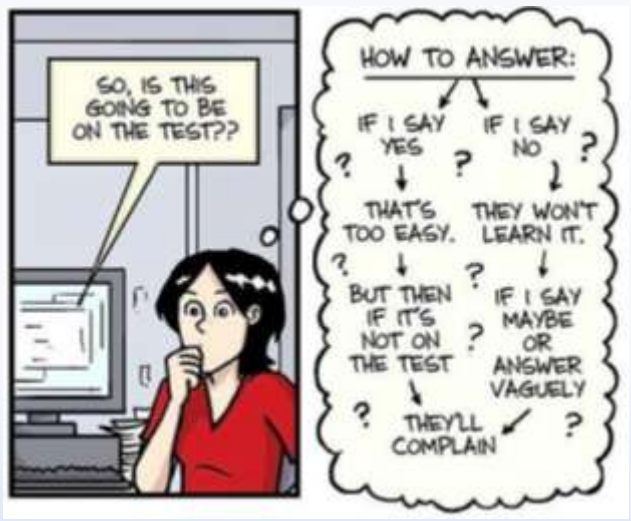
CIS 613:
Final Review

1



2

3

Note: the Final *is* cumulative.

- You will be responsible for all the information on the previous exam, especially:
  - General concepts of
    - Boundary Value Analysis,
    - Equivalence Class Testing, and
    - Decision Table-based Testing
  - **Path-Based Testing**
  - **Code Coverage Testing**
- However, the exam will *focus* on the new material.

4

Input Boundary Value Testing

- Test values for variable x, where
  - $a \leq x_1 \leq b$, and
  - x(min), x(min+), ... x(max) are the names from the T tool.

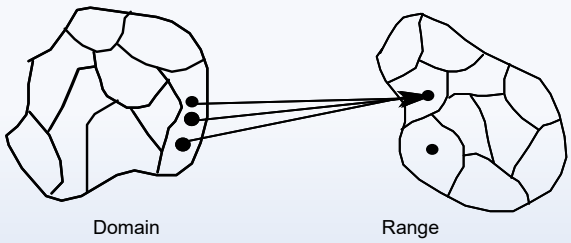
5

Equivalence Partitions

Define a relation R on the input domain D as:

$$\forall x, y \in D, xRy \text{ iff } F(x) = F(y),$$

where F is the program function.



Equivalence Relations (e.g., R) treat inputs the *same*.

6

Example: Last Day of the Month (Extended Entry) Test Cases

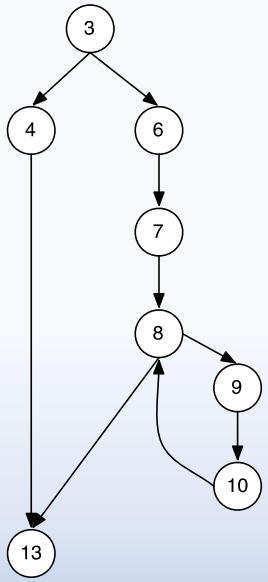
	R_1	R_2	R_3	R_4
c1,2,3. month in	M1	M2	M3	M3
c4. leap year?	—	—	T	F
a1. last day = 30	x			
a2. last day = 31		x		
a3. last day = 28				x
a4. last day = 29			x	

	Month	Year	Last Day
Test Case for R_1	M1	Regular Year	30
Test Case for R_2	M2	Regular Year	31
Test Case for R_3	M3	Leap Year	29
Test Case for R_4	M3	Regular Year	28

7

Basis Path Testing

```
1.  int factorial(int n) {
2.      int i, fact;
3.      if(n < 0) {
4.          fact = 0;
5.      } else {
6.          i = 1;
7.          fact = 1;
8.      while(i <= n) {
9.              fact = fact * i;
10.             i = i + 1;
11.         }
12.     }
13.  return fact;
14. }
```



8

Program Graphs: In Class Exercise

```
 -  // Method returns the greatest common divisor of a and b
 -
 -  private static int gcd (int a, int b) {
 -
 1     if(a < 0) {
 2        a = Integer.MIN_VALUE;
 3     } else if(b < 0) {
 4        a = Integer.MIN_VALUE;
 5     } else {
 6        while (b != 0) {
 7           int temp = b;
 8           b = a % b;
 9           a = temp;
 -        }
 -     }
 -
10     return a;
 -  }
```

9

Code Coverage Testing

10

Coverage Based Testing

Design enough test cases such that:

- **Line coverage**: every line in a program is executed at least once.
- **Branch coverage**: each decision/branch has a *true* and *false* outcome at least once.
- **Condition coverage**: each condition in a decision/branch takes all possible outcomes at least once.
- **Branch/Condition coverage**: each condition in a decision takes on all possible outcomes at least once, *and* each decision takes on all possible outcomes at least once.

11

Coverage Based Testing

Design enough test cases such that:

- **Multiple condition coverage**: all possible combinations of condition outcomes in each decision are invoked at least once.
- **MC/DC**: MCC *and* each condition is show to independently affect the outcome of the decision.
- **Path coverage**: all execution paths are executed at least once:
  - A path is a unique sequence of branches from entry to exit.
  - Loops introduce an unbounded number of paths making path coverage criterion not practical.
  - Some paths are impossible to exercise due to relationships of data.
  - The number of executions of a loop may depend on the input variables and hence may not be possible to determine.

12

In Class Exercise

- **Line coverage**: every line in a program is executed at least once.
- **Branch coverage**: each decision/branch has a *true* and *false* outcome at least once.
- **Condition coverage**: each condition in a decision/branch takes all possible outcomes at least once.
- **Branch/Condition coverage**: each condition in a decision takes on all possible outcomes at least once, *and* each decision takes on all possible outcomes at least once.
- **Multiple condition coverage**: all possible combinations of condition outcomes in each decision are invoked at least once.

```
int function(int a, int b, int c)  {

   int branchCount = 0;
   if((a != 10) || (b == 10)) {
      branchCount++;
   }

   if((c == 10) && (b >= 10)) {
      branchCount++;
   }

   return branchCount++;
}
```

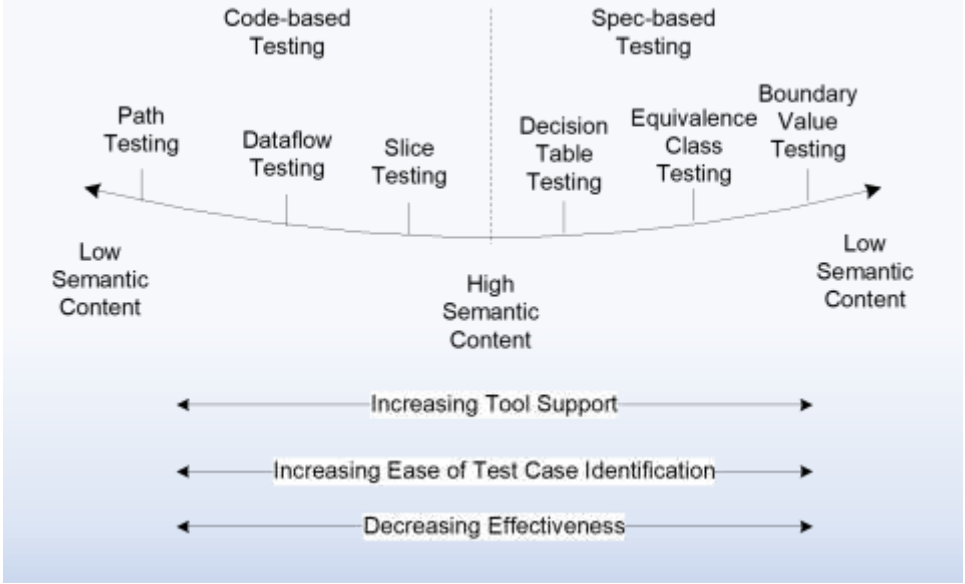
13

Decision Table & Dataflow Testing

- Condensation, How to select tests from reduced decision table
- DU-Paths
- DC-Paths
- DEF, USE identification
- Slice (Forward & Backward)
- Debugging

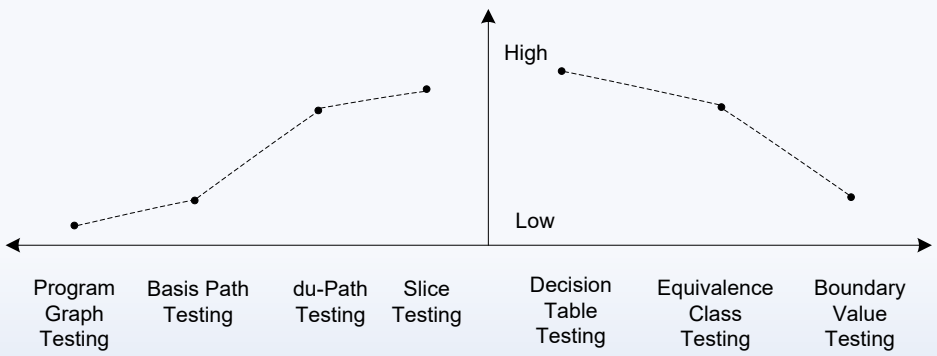
14

The Test Method Pendulum



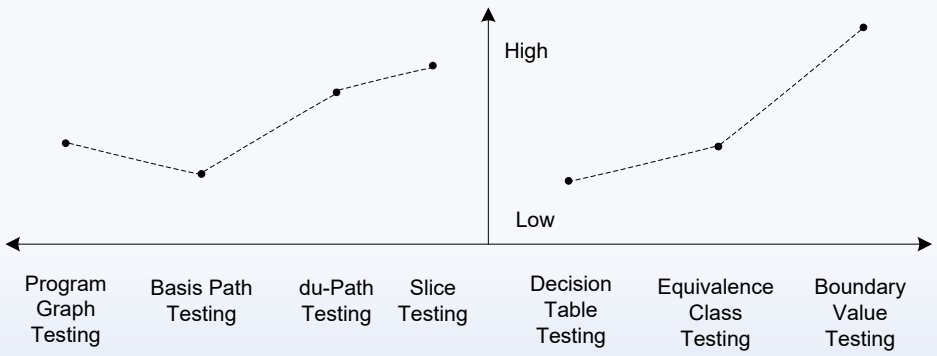
15

Test Case Identification Effort



16

Number of Test Cases



17

Lifecycle-based Testing

- Waterfall (& V-Model)
- Iterative
- Rapid Prototyping
- Executable Specification
- Agile Methods

18

Model-Based Testing

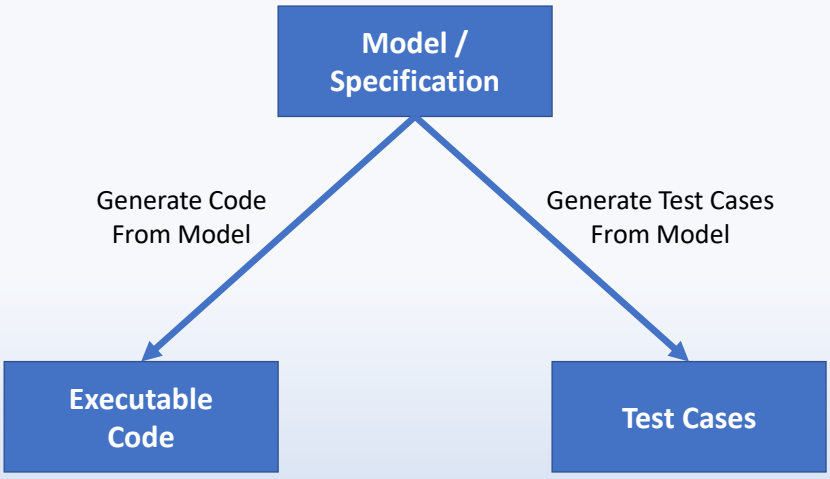
19

Model-Based Testing Procedure

- Understand application to be modeled
- Identify behavioral issues in the application
- Select *appropriate* model
  - this may depend on staff talent
  - or existing development tools
  - or other immutable requirements
- Model the application
- Use an engine to “execute” the model, thereby identifying
  - interesting scenarios
  - automatically generated test case (skeletons)

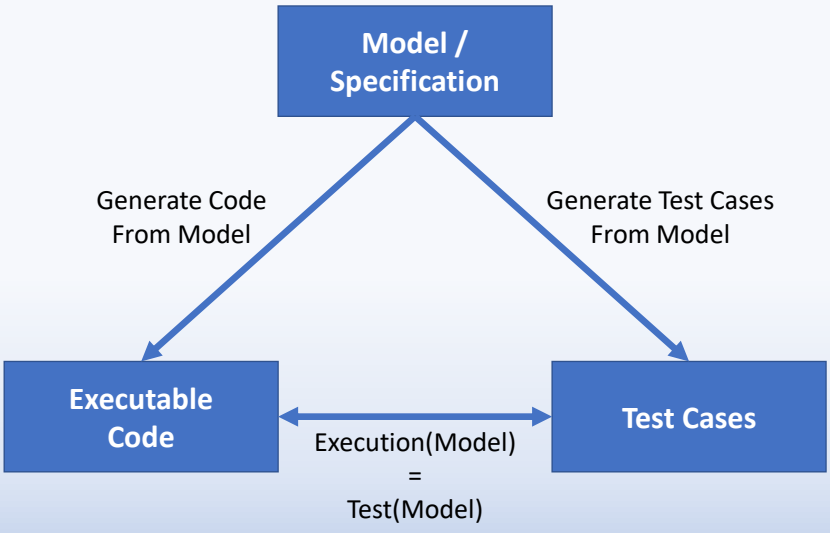
20

Executable Specification vs Model-Based Development vs Model-Based Testing?



21

Executable Specification vs Model-Based Development vs Model-Based Testing?



22

Mutation Testing

- What is the purpose of testing?
- How do these combine?
  - Competent Programmer Hypothesis
  - Coupling Effect
- Given test cases, and a mutation – what is the mutation score?
- Given a mutation, identify (define) a test that could 'kill' it

23

Definitions

- **Mutation**: Syntactic change in a program
- **Mutation Analysis**: Assessing the quality of a test suite
- **Mutation Testing**: Improving the test suite using mutants
- **Mutant**: Change in a program to cause a fault
- **Killed Mutants (Dead Mutants)**: Mutants detected by test suite
- **Live Mutants**: Mutants not detected by the test suite

Mutation Score = Killed Mutants / Total Mutants

24

Genetic Algorithms & Testing

25

What *is* a Genetic Algorithm

- Basic idea: Simulate natural selection, where the population is composed of candidate solutions.
- Genetic algorithms are a randomized heuristic search strategy.
- Focus is on evolving a population from which fit and diverse candidates can emerge via mutation and crossover.
- Genetic Operators:
  - Mutation emphasizes diversity
  - Crossover emphasizes diversity *and* fitness
  - Selection emphasizes fitness

26

Genetic Algorithm

1. **Generate a population** of candidate solutions
2. **Evaluate**: Calculate the "fitness" of each member of the population
3. **Selection**: Probabilistically choose a subset of the population to survive
4. **Variation**: Permute individuals using genetic operators (crossover, mutation)
5. **Update Population**: Form next generation using selection and variation.
6. **Termination**: Stop when maximum fitness is achieved, or a specified number of generations.

27

Crossover

Crossover combines inversion and recombination:

- $Parent_1$ = (3, 5, 7, 2, 1, 6, 4, 8)
- $Parent_2$ = (2, 5, 7, 6, 8, 1, 3, 4)

- Child = (2, 5, 7, 2, 1, 6, 3, 4)

Will the crossover operator always generate a valid child that is better (i.e., more fit) than the parents?

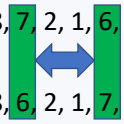
A. The child will **always be valid** and **always more fit** than the parents
B. The child will **always be valid** and **not always more fit** than the parents
C. The child will **not always be valid** and **always more fit** than the parents
D. The child will **not always be valid** and **not always more fit** than the parents

28

Mutation

Mutation involves (in this case) reordering the list of cities:

- Before = (5, 8, 7, 2, 1, 6, 3, 4)
- After = (5, 8, 6, 2, 1, 7, 3, 4)



Will this mutation operator always generate a valid sequence that is better (i.e., more fit) than the unmutated version?

A. The sequence will **always be valid** and **always more fit** than before
B. The sequence will **always be valid** and **not always more fit** than before
C. The sequence will **not always be valid** and **always more fit** than before
D. The sequence will **not always be valid** and **not always more fit** than before

29

Issues for Genetic Algorithms

- Choosing basic implementation issues:
  - representation
  - population size, mutation rate, ...
  - selection, deletion policies
  - crossover, mutation operators
- Termination Criteria
- Performance, scalability
- Solution is only as good as the evaluation function (often hardest part)

How do these relate to testing?

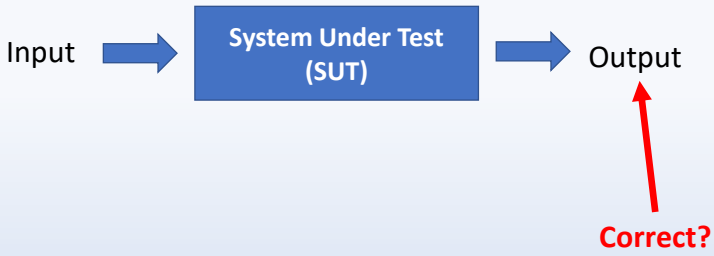
30

Metamorphic Testing

31

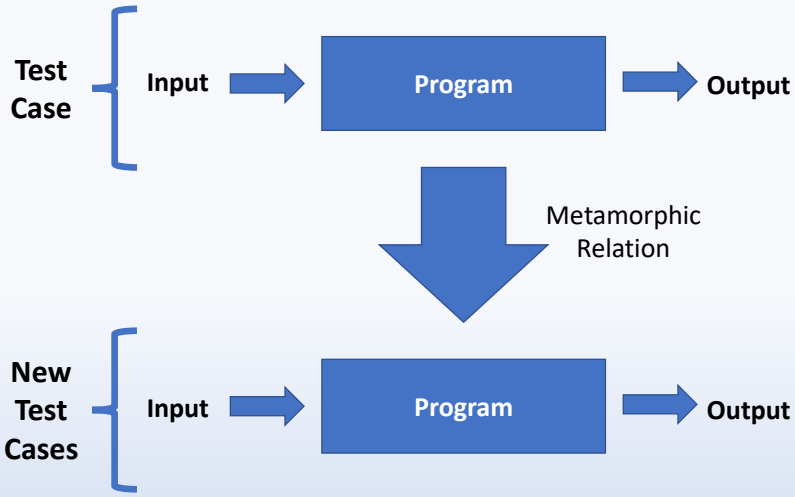
Test Oracle

Test Oracle: Mechanism to decide whether a test output is correct or not.



32

Metamorphic Relation



33

Guidelines for Metamorphic Testing

- Metamorphic testing requires good knowledge of the problem domain.

34

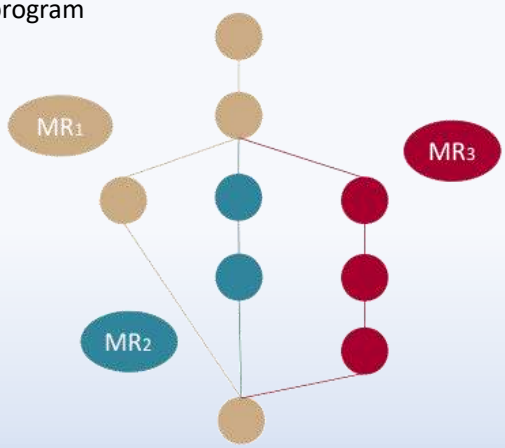
Guidelines for Metamorphic Testing

- Different metamorphic relations can have different fault-detection capability

35

Guidelines for Metamorphic Testing

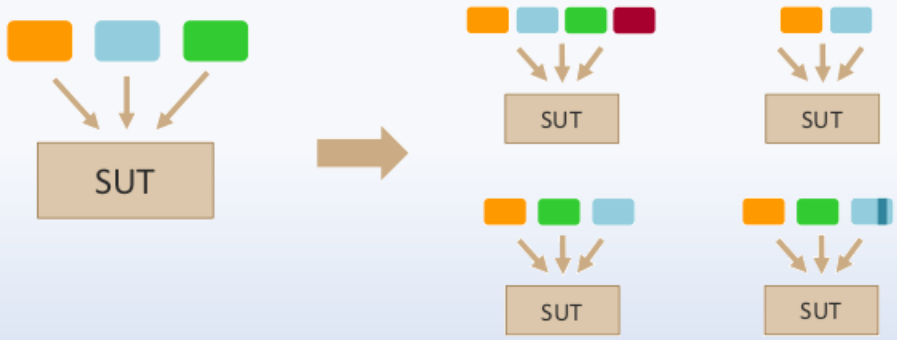
- Metamorphic relations should be diverse to they exercise different parts of the program



36

Guidelines for Metamorphic Testing

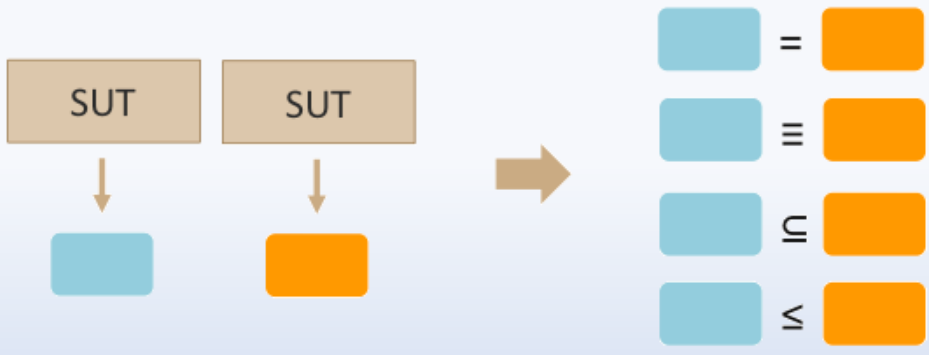
- Two common approaches for the construction of metamorphic relations are **input-driven** and output driven.



37

Guidelines for Metamorphic Testing

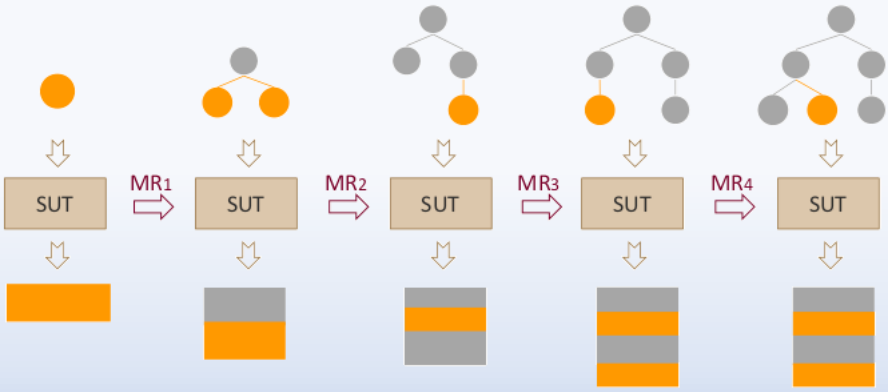
- Two common approaches for the construction of metamorphic relations are input-driven and **output driven**.



38

Guidelines for Metamorphic Testing

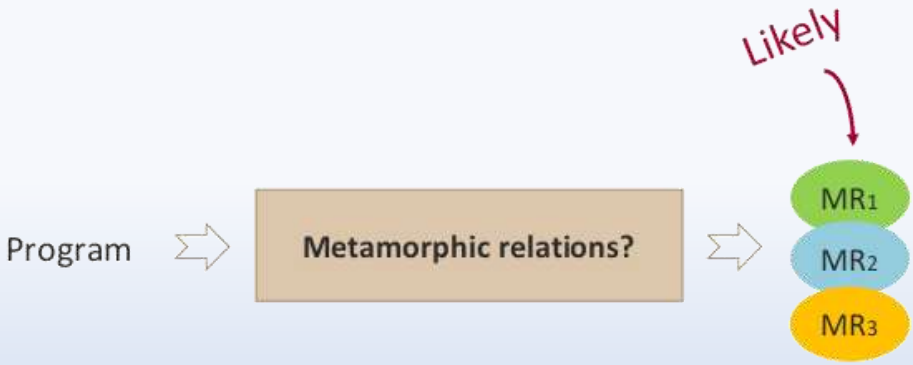
- Metamorphic relations can be combined



39

Guidelines for Metamorphic Testing

- The automated discovery of metamorphism relations seems feasible in certain domains



Also would be a great Masters Thesis!

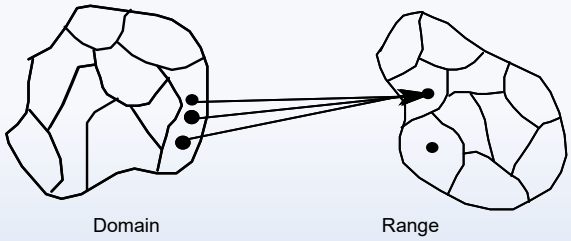
40

Equivalence Partitions vs Metamorphic Relations?

Define a relation R on the input domain D as:

$$\forall x, y \in D, xRy \text{ iff } F(x) = F(y),$$

where F is the program function.



Equivalence Relations (e.g., R) treat inputs the *same*.

How do equivalence relations and metamorphic relations relate?

41

Integration Testing

42

Testing Level Assumptions and Objectives

	Assumptions	Goals
Unit	• All other units are correct • Compiles correctly	• Correct unit function • Coverage metrics satisfied
Integration	• Unit testing complete	• Interfaces correct • Correct function across units • Fault isolation support
System	• Integration testing complete • Tests occur at port boundary	• Correct system functions • Non-functional requirements tested • Customer satisfaction

43

Approaches to Integration Testing

("source" of test cases)

- Functional Decomposition (most commonly)
  - Top-down
  - Bottom-up
  - Sandwich
  - "Big bang"
- Call graph
  - Pairwise integration
  - Neighborhood integration
- Paths
  - MM-Paths

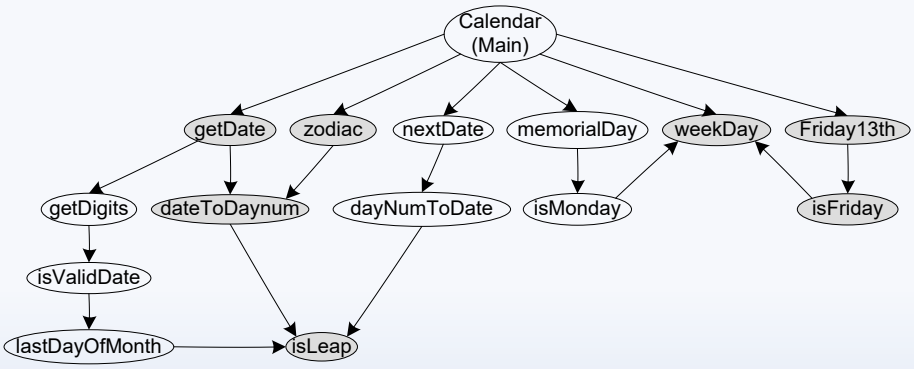
44

Neighborhood Integration

- The neighborhood (or radius 1) of a node in a graph is the set of nodes that are one edge away from the given node.
- This can be extended to larger sets by choosing larger values for the radius.
- Stub and driver effort is reduced.

45

Two example neighborhoods for Neighborhood Integration



What is the neighborhood radius for the two neighborhoods?

A. Radius of 1
B. Radius of 2
C. Radius of 3

46

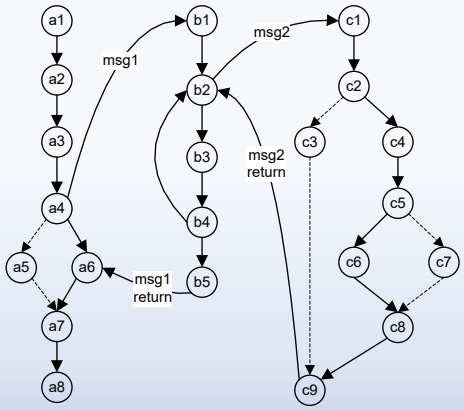
New and Extended Definitions

- A **source node** in a program is a statement fragment at which program execution begins or resumes.
- A **sink node** in a unit is a statement fragment at which program execution terminates.
- A **module execution path** is a sequence of statements that begins with a source node and ends with a sink node, with no intervening sink nodes.
- A **message** is a programming language mechanism by which one unit transfers control to another unit, and acquires a response from the other unit.

47

MM-Path Definition & Example

- An MM-Path is an interleaved sequence of module execution paths and messages.
- An MM-Path across three units:



48

Comparison of Integration Testing Strategies

	Ability to test interfaces	Ability to test co-functionality	Fault isolation and resolution
Functional Decomposition	acceptable, but can be deceptive (phantom edges)	limited to pairs of units	Identifies Unit
Call Graph	acceptable	limited to pairs of units	Identifies Unit
MM-Path	excellent	complete	Identifies Unit & Path

49

System Testing

50

FSA Path Probabilities

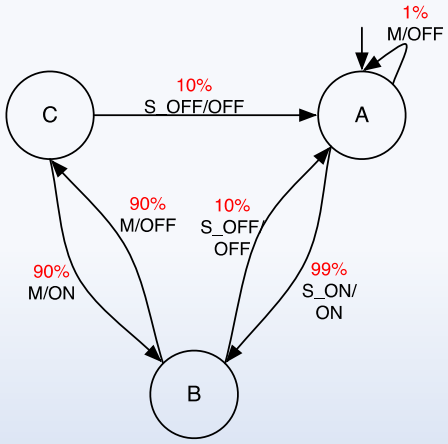
Automatic Light Switch Finite State Machine

Thread (State, Action):

- A, Turn switch on (S_ON)
- B, Leave room (M)
- C, Enter room (M)
- B.

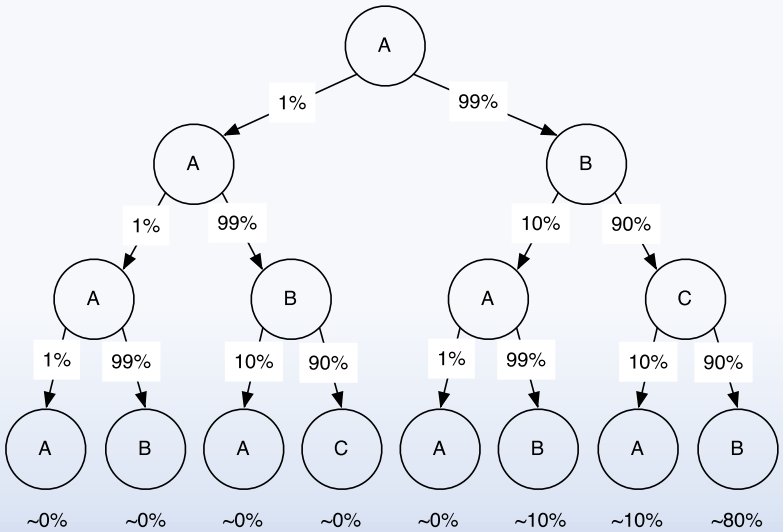
Probability (State, Action = %):

- A, S_ON: 99%
- B, M: 90%
- C, M: 90%
- B = 99%*90%*90% = ~80%



51

Given the likelihood, add the risk!



52

Risks in Software

- A risk is an unwanted event that has negative consequences (often due to incorrect estimation)
- Distinguish risks from other project events
  - **Risk impact**: the loss associated with the event
  - **Risk probability**: the likelihood that the event will occur
- Quantify the effect of risks
  - **Risk exposure** = (risk probability) x (risk impact)

53

Risk-Based Testing

Probability \ Impact	Negligible	Marginal	Critical	Catastrophic
Certain	High	High	Extreme	Extreme
Likely	Moderate	High	High	Extreme
Possible	Low	Moderate	High	Extreme
Unlikely	Low	Low	Moderate	Extreme
Rare	Low	Low	Moderate	High

54

Model Checking & Race Conditions

55

Model Checking

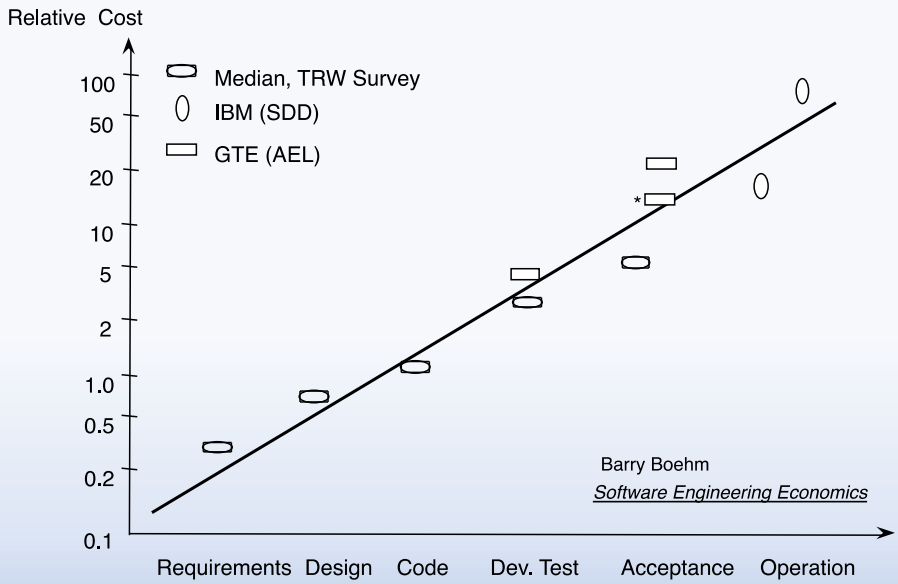
- Testing Concurrency
- Temporal Logic
- Connection to Model-Based Testing
- Issues of Injecting Faults when Modeling

56

Technical Reviews

57

Fault Resolution Costs



58

Cost/Benefit Reports

- Inspections can be 5% to 15% of total project cost
- "Jet Propulsion Laboratory estimated a net savings of $7.5 million from 300 inspections performed on software they produced for NASA."
- "another company reports annual savings of $2.5 million"
  - Cost to fix a defect found by inspection: $146
  - Cost to fix a defect found by customer: $2900
  - Cost/Benefit ratio is 0.0503

Karl E. Wiegers "Improving Quality Through Software Inspections"

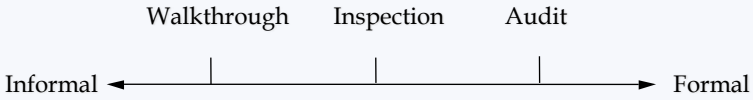
59

Review Roles & Purpose

- Roles
  - Responsible designer (aka Producer)
  - Recorder
  - Review leader
  - Reviewer
- To assure success, reviews must have
  - Reviewer credibility
  - Process credibility
  - Management and technical commitment

60

Types of Reviews



- **Formality**
  - Informal reviews often involve just 2 engineers, short duration, no documentation, low credibility
  - formal reviews involve commitment, preparation and responsibility
- **Internal reviews**: very effective
- **External reviews**: carefully planned, rehearsed
  - seldom very effective (at finding faults!)
  - easily degenerate into Dog and Pony shows

61

Walkthroughs

- Responsible designer is review leader
- Most common form of review
- Most questions stimulated by presenter
- Frequently a presentation
- Effectiveness depends on…
  - real goal of responsible designer
  - preparation of reviewer(s)

62

Technical Inspections

- Responsible designer is not the review leader
- Formal process
- Very participative
- Advance preparation required
- Centers on checklist

- Effectiveness depends on …
  - reviewer preparation
  - nature of checklist

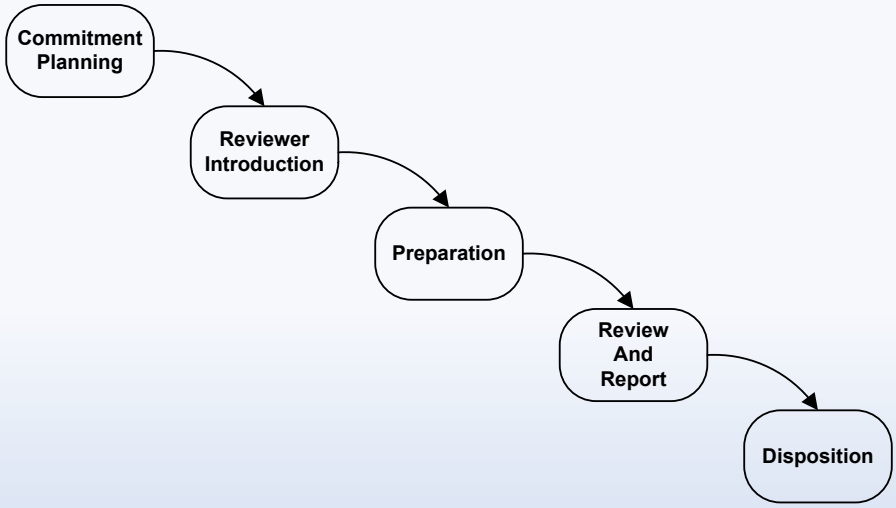
63

Walkthroughs vs Inspections

	Walkthrough	**Inspection**
Coverage	Broad, Superficial	Deep, Thorough
"Driver"	Responsible Designer	Review Leader
Preparation Time	Low	High
Formality	Informal	Formal
Effectiveness	Low	High

64

Industrial-Strength Inspection Phases



65

Review Etiquette

- Reviewers should be prepared.
- Review the product, not the producer.
- Stick to technical issues.
- Be constructive; reviews are not the place for
  - negative comments
  - "pats on the back“
- Identify issues, don't resolve them.
- Provide minor comments (e.g., spelling corrections) in writing to the producer at the end of the review meeting.
- Avoid discussions of style.
- Record all issues in public.
- Above all: Don’t waste time!

66

Worst Case Review

- Producer picks friendly reviewers
- Short lead time
- No approved preparation time
- Deliverable not frozen
- Postponed twice
- Some reviewers absent
- Top designers excused (from reviews and reviewing)
- No checklist
- No issue list, no action items
- Page-by-page agenda
- Faults are resolved
- Coffee and lunch breaks needed
- Reviewers leave and return
- Producer's supervisor is review leader
- "cast of thousands"

67

Desirable Review Culture

- Producers do not dread reviews
- Reviewers have approved preparation time
- Materials delivered with sufficient lead time
- Trained review leaders
- Reviewers perceive reviews as productive
- Management perceives reviews as productive
- Review meetings have high priority
- Checklists are actively maintained
- Top designers are frequent reviewers
- Reviewer effectiveness is considered during performance evaluation
- Management actively supports reviews
- Management uses results of reviews

68

Exploratory Testing

69

What is exploratory testing?

- Explore: to investigate the unknown
- "The opposite of scripted testing"—James Bach
- More than ad hoc testing
- Shares some pros and cons with Special Value Testing
- Depends on the abilities of the exploratory tester
  - Curiosity
  - Ingenuity
  - Time and tools
- A discovery process
- A learning process in which the results of one test suggest additional tests

70

Testing Excellence

71

Top 10 Best Practices for Software Testing Excellence

1. Model-Driven Agile Development
2. Careful Definition and Identification of Levels of Testing
3. System-Level Model-Based Testing
4. System Testing Extensions
5. Incidence Matrices to Guide Regression Testing
6. Use of MM-Paths for Integration Testing
7. Intelligent Combination of Specification-Based and Code-Based Unit Level Testing
8. Code Coverage Metrics Based on the Nature of Individual Units
9. Exploratory Testing During Maintenance
10. Test-Driven Development

72

Mapping Best Practices to Application Types

Best Practice	Mission Critical	Time Critical	Legacy Code
Model-Driven Development	x		
Careful Definition and Identification of Levels of Testing	x	x	x
System-Level Model-Based Testing	x		
System Testing Extensions	x		
Use of MM-Paths for Integration Testing	x		
Intelligent Combination of Specification-Based and Code-Based Unit Level Testing	x		x
Code Coverage Metrics Based on the Nature of Individual Units	x		
Exploratory Testing During Maintenance			x
Test-Driven Development		x	

73

Test Maturity Model

- A Spin-off of Capability Maturity Model
- 5 Levels
  - **Initial**: ad hoc, no quality standard
  - **Definition**: possible defined test process. Testing begins when project development ends.
  - **Integration**: testing integrated with development. Possile independent test teams.
  - **Management and measurement**: (name says most of it), quality criteria in place
  - **Optimization**: tools incorporated into process, emphasis shifts to defect prevention

74

Any other questions or review items?

75